Université Saad Dahlab - Blida1

Faculté des Sciences - Département d'Informatique

Semestre/Année : S2 2025/2026 **Classe** : M1 DS&NLP

Module : NoSQL pour le Big Data **Enseignant** : Dr. B. BENCHARIF

Travaux Pratiques 04 MapReduce et Analyse de Données Massives avec MongoDB

Base de données : `universite`

Collections utilisées :

`transactions_tp4 (dataset simulé)`

`comments (analyse de texte)`

1. Introduction et Objectifs

1.1. Contexte Théorique

Dans l'ère du **Big Data**, le traitement des données ne peut plus s'appuyer sur des méthodes transactionnelles classiques. Lorsque le volume de données dépasse la capacité mémoire d'une seule machine, il devient nécessaire d'adopter des paradigmes de **calcul distribué**.

Le modèle **MapReduce**, introduit par Google en 2004, a révolutionné ce domaine. Il repose sur le principe du "*Divide and Conquer*" (Diviser pour régner) :

- **Map** : Distribuer le travail en transformant les données brutes en paires Clé/Valeur.
- **Shuffle** : Réorganiser et regrouper les données par clé à travers le réseau.
- **Reduce** : Agréguer les résultats intermédiaires pour obtenir une synthèse finale.

Bien que MongoDB ait introduit l'**Aggregation Framework** (Pipeline) pour offrir une alternative plus rapide et plus intuitive (codée en C++ natif), la compréhension de MapReduce reste indispensable pour tout Data Scientist. C'est la base conceptuelle de systèmes massifs comme **Apache Hadoop** et **Apache Spark**.

1.2. Objectifs du TP

Ce travail pratique vise à confronter l'étudiant à la réalité du traitement de données à grande échelle. À l'issue de cette séance, vous serez capable de :

- **Maîtriser le paradigme fonctionnel** : Apprendre à modéliser un problème analytique complexe sous forme de fonctions *Mapper* et *Reducer*.
- **Évaluer l'efficacité algorithmique** : Comprendre pourquoi le moteur natif d'agrégation de MongoDB surpasse le moteur JavaScript de MapReduce dans la majorité des cas d'usage.

- **Ingénierie de données textuelles** : Appliquer ces concepts sur des données non structurées (Text Mining) pour extraire des connaissances (Fréquence de mots/Word Count) à partir de milliers de commentaires.
- **Optimisation des ressources** : Apprendre à utiliser des mécanismes comme `allowDiskUse` pour traiter des jeux de données dépassant la RAM disponible.

À la fin de ce TP, l'étudiant sera capable de :

- ✓ Décomposer un problème d'agrégation en étapes **Map**, **Shuffle** et **Reduce**.
- ✓ Implémenter et comparer les performances des deux méthodes.
- ✓ Manipuler des données textuelles non structurées (Word Count).

2. Liste des Fichiers Annexes

Avant de commencer, assurez-vous d'avoir les fichiers suivants (fournis en pièces jointes) :

`concept_mapreduce.py` : Simulation pédagogique du fonctionnement de MapReduce.

`generate_data.py` : Script pour générer 10 000 transactions dans MongoDB.

`benchmark_tp4.js` : Script JavaScript pour comparer MapReduce et Aggregation.

`benchmark_comments_tp4.js` : Script JavaScript pour comparer MapReduce et Aggregation.

`wordcount_comments.py` : Analyse textuelle avancée sur la collection `comments`.

3. Comprendre le Concept (Simulation Python)

Avant d'utiliser la base de données, étudions comment les données sont transformées.

Action : Exécutez le fichier `concept_mapreduce.py`.

Analyse : Observez comment chaque mot est transformé en paire `(clé, 1)`, regroupé par le système (**Shuffle**), puis additionné (**Reduce**).

4. Partie Pratique : Implémentation et Benchmark sous MongoDB

Cette section présente la mise en œuvre technique et l'analyse comparative des performances entre les deux méthodes de traitement de données massives proposées par MongoDB.

Étape 1 : Préparation de l'environnement

L'infrastructure de test a été configurée comme suit :

1. **Instance MongoDB :** Lancement du serveur (mongod) pour accueillir les données.
2. **Génération des données :** Exécution du script `generate_data.py`. Ce script a permis de peupler la collection `transactions_tp4` avec un volume significatif de documents afin de rendre le benchmark pertinent.

Étape 2 : Comparaison MapReduce vs Aggregation Framework

L'objectif est de calculer le montant total des ventes et le nombre de transactions par catégorie de produit.

4.1. Résultats du Benchmark

Après exécution du script `benchmark_tp4.js` + `benchmark_comments_tp4.js` via le shell **mongosh**, nous avons relevé les mesures suivantes :

Méthode	Temps d'exécution (ms)	Complexité du code (Lignes)
MapReduce		
Aggregation Framework		

4.2. Analyse de performance : Pourquoi l'Aggregation est-elle plus rapide ?

Bien que les deux méthodes produisent un résultat identique, l'**Aggregation Framework** surpasse MapReduce pour plusieurs raisons techniques :

- **Exécution Native :** L'Aggregation est écrite en **C++** et s'exécute directement dans le noyau de MongoDB, tandis que MapReduce repose sur l'interprétation de code **JavaScript** (via le moteur SpiderMonkey), ce qui induit une latence importante.
- **Optimisation du Pipeline :** MongoDB utilise un optimiseur de requêtes interne pour réorganiser les étapes du pipeline (ex: `$match` avant `$group`) afin de minimiser le flux de données.
- **Évitement de la Sérialisation :** MapReduce nécessite de convertir les données BSON en objets JSON pour JavaScript, puis de refaire l'opération inverse, contrairement au framework d'Aggregation qui travaille nativement.

Étape 3 : Analyse Textuelle (Word Count sur Big Data)

Nous avons procédé à une analyse de fréquence de mots sur la collection `comments`, qui simule un volume de données massif (milliers de documents).

Procédure technique :

- **Implémentation :** Complétion du script `wordcount_comments.py` en utilisant un pipeline composé généralement de `$project` (split des phrases), `$unwind` (séparation des mots) et `$group` (comptage).
- **Gestion de la mémoire :** Utilisation impérative du paramètre `allowDiskUse: true`.
    - **Note :** Cette option est cruciale car elle permet à MongoDB de déborder sur le disque dur si une étape du pipeline dépasse la limite de **100 Mo** de RAM, évitant ainsi une interruption du script sur de gros volumes.

Travail demandé :

Identifiez les 10 mots les plus fréquents. Utilisez le paramètre `allowDiskUse: true` pour gérer le volume de données.

5. Missions Pratiques : "Le Challenge des Data Engineers"

Cette phase finale vous place dans une situation réelle d'ingénierie de données. Vous allez travailler sur des jeux de données volumineux issus de **Kaggle**. Chaque groupe est investi d'une mission spécifique visant à transformer la donnée brute en information stratégique.

Consignes Générales

- **Exploration :** Analysez d'abord la structure de vos documents avec la commande `db.collection.findOne()`.
- **Implémentation :** Développez votre solution en utilisant le **Pipeline d'Agrégation**.
- **Benchmark :** Comparez, si possible, le temps d'exécution avec une approche **MapReduce**.
- **Optimisation :** Activez le paramètre `allowDiskUse: true` pour gérer la volumétrie sur disque.

Mission A : Pôle "Insights & Tendances"

Objectif : Identifier les thématiques dominantes ou les produits phares au sein d'un flux massif de données.

- **Tâches à réaliser :**
  1. Filtrer les données pertinentes selon des critères temporels ou catégoriels (`$match`).
  2. Extraire et compter les occurrences de mots ou d'identifiants (`$group` + `$sum`).
  3. Trier les résultats pour extraire le "Top 10" des tendances (`$sort` + `$limit`).
- **Défi Technique :** Comment assurer la précision du comptage malgré la présence de données redondantes ?

Mission B : Pôle "Qualité & Transformation"

Objectif : Nettoyer une collection contenant des données hétérogènes (types mixtes, erreurs de saisie) pour la rendre exploitable.

- **Tâches à réaliser :**
  1. Convertir les champs textuels en types numériques ou dates (`$convert` ou `$toDouble`).
  2. Calculer des indicateurs de synthèse (moyennes, écarts-types) sur les données nettoyées.
  3. Sauvegarder la vue propre dans une nouvelle collection permanente via l'étape `$out`.
- **Défi Technique :** Comment gérer les documents dont les champs sont manquants (`null`) sans faire planter le pipeline ?

Mission C : Pôle "Structure & Corrélation"

Objectif : Croiser les informations provenant de deux collections distinctes pour obtenir une vision multidimensionnelle.

- **Tâches à réaliser :**
  1. Effectuer une jointure entre la collection principale et une collection de référence (`$lookup`).
  2. Aplatir les résultats de la jointure pour effectuer des calculs statistiques (`$unwind`).
  3. Regrouper les données par zone géographique ou par segment utilisateur.
- **Défi Technique :** Évaluer l'impact de la jointure sur le temps d'exécution par rapport à une requête simple.

Livrables Attendus par Groupe

À la fin de la session, chaque groupe devra présenter :

1. **Le Pipeline final :** Le code JavaScript ou Python utilisé pour résoudre la mission.
2. **Le Rapport de Performance :** Le temps d'exécution relevé et les éventuels blocages mémoire rencontrés.
3. **L'Analyse Critique :** Pourquoi avez-vous privilégié une étape plutôt qu'une autre dans votre pipeline ?